

工具系统

Hermes Agent 工具系统深度分析报告

目录

- 1. 概述
 - 1.1 分析范围
 - 1.2 工具全景图
- 2. 核心架构总览
 - 2.1 导入链设计
 - 2.2 ToolEntry 数据结构
- 3. 工具注册中心 (ToolRegistry)
 - 3.1 单例模式
 - 3.2 核心方法分析
 - 3.3 查询接口
- 4. 工具集系统 (Toolsets)
 - 4.1 分层结构设计
 - 4.2 resolve_toolset() 的递归解析
 - 4.3 平台特定工具集
 - 4.4 插件工具集的动态注册
- 5. 工具编排层 (model_tools.py)
 - 5.1 工具发现机制
 - 5.2 异步桥接的三层设计
 - 5.3 Schema 动态修补
- 6. 工具调用解析器 (Tool Call Parsers)
 - 6.1 问题背景
 - 6.2 解析器注册机制
 - 6.3 Hermes Parser 示例
- 7. MCP 协议支持
 - 7.1 架构定位
 - 7.2 MCP 客户端核心设计
 - 7.3 安全设计
 - 7.4 Sampling 机制
 - 7.5 重连与容错
- 8. 插件系统
 - 8.1 插件发现三源
 - 8.2 PluginContext 门面
 - 8.3 生命周期钩子
 - 8.4 禁用机制
- 9. 代码执行沙箱
 - 9.1 沙箱工具集注入
 - 9.2 多种执行后端
- 10. 生命周期钩子与拦截器
 - 10.1 Agent Loop 工具拦截
 - 10.2 读写工具连续性检测
- 11. 工具集分发系统
 - 11.1 RL 训练中的工具概率分布
- 12. 架构决策中的权衡
 - 12.1 单例 Registry vs 依赖注入
 - 12.2 静默失败 vs 严格检查
 - 12.3 check_fn 粒度
 - 12.4 工具 Schema 的版本管理
- 13. 安全设计分析
 - 13.1 纵深防御
 - 13.2 凭证清理的局限性

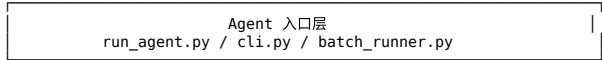
1. 概述

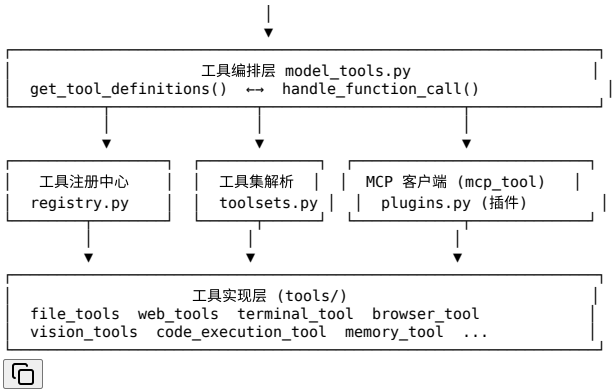
1.1 分析范围

覆盖以下核心子系统：

子系统	文件路径	核心职责
工具注册中心	tools/registry.py	工具的注册、存储与分发
工具集系统	toolsets.py	工具分组与动态组合
工具编排层	model_tools.py	工具发现、Schema 生成与调用分发
工具调用解析器	environments/tool_call_parsers/	多模型格式的解析
MCP 客户端	tools/mcp_tool.py	外部 MCP 服务器集成
MCP 服务器	mcp_serve.py	暴露 Agent 为 MCP 服务
插件系统	hermes_cli/plugins.py	第三方扩展机制
代码执行沙箱	tools/code_execution_tool.py	受控代码运行环境
工具集分发	toolset_distributions.py	RL 训练中的工具概率分布

1.2 工具全景图





2. 核心架构总览

2.1 导入链设计

Hermes Agent 的工具系统采用**零耦合中心化注册**模式，工具模块在导入时自我注册，整个链路的循环导入安全设计如下：

```
tools/registry.py      ← 无任何依赖，是整个系统的叶节点
    ↑
tools/*.py             ← 仅依赖 registry，在 import 时调用 register()
    ↑
model_tools.py         ← 依赖 registry + toolsets
    ↑
run_agent.py / cli.py  ← 依赖 model_tools，是入口节点
```

Copy icon

2.2 ToolEntry 数据结构

```
class ToolEntry:
    __slots__ = (
        "name",          # 工具唯一名称
        "toolset",        # 所属工具集
        "schema",         # OpenAI 格式 Schema
        "handler",        # 处理函数
        "check_fn",       # 可用性检查函数
        "requires_env",   # 所需环境变量列表
        "is_async",       # 是否为异步工具
        "description",    # 工具描述
        "emoji",          # UI 显示 emoji
    )
```

Copy icon

3. 工具注册中心（ToolRegistry）

3.1 单例模式

```
registry = ToolRegistry() # 模块级单例
```

Copy icon

使用模块级单例而非依赖注入（DI）容器，是语言习惯和实用主义的平衡。在 CLI 工具场景中，DI 容器的复杂度远大于收益；但对于测试和 MCP 远程调用场景，这埋下了单例状态的隐患。

3.2 核心方法分析

register() — 工具注册

```
def register(
    self, name, toolset, schema, handler,
    check_fn=None, requires_env=None, is_async=False,
    description="", emoji=""
):
```

Copy icon

Note

设计洞察：

- **幂等覆盖**：同名的重复注册会发出 WARNING 而非抛出异常，允许 MCP 动态重新注册工具。这在 MCP 服务器工具列表变化时（notifications/tools/list_changed）至关重要。
- **toolset check_fn 合并**：每个 toolset 只存储第一个注册工具的 check_fn，后续同 toolset 的 check_fn 被静默忽略。这意味着同一 toolset 的多个工具必须共享同一个可用性检查逻辑，设计者需要确保工具与 toolset 的 check_fn 语义一致。
- **is_async 标记**：工具自行声明异步能力，registry 在分发时透明桥接同步/异步边界。这一标记避免了运行时 introspect 的开销。

get_definitions() — Schema 聚合与过滤

```
def get_definitions(self, tool_names: Set[str], quiet: bool = False) -> List[dict]:
```

Copy icon

这是工具系统与 LLM API 的接口。关键行为：

1. **check_fn 缓存**：check_results 字典在单次调用内缓存 check_fn 结果，避免同一检查函数被多次执行。
2. **总是补全 name 字段**：即使 Schema 中缺少 name，也会用 entry.name 兜底，防止 LLM 收到无名的工具定义。
3. **返回 OpenAI format**：{"type": "function", "function": {...}} 格式直接对接 OpenAI API。

dispatch() — 工具执行

```
def dispatch(self, name: str, args: dict, **kwargs) -> str:
    entry = self._tools.get(name)
    if not entry:
        return json.dumps({"error": f"Unknown tool: {name}"})
    try:
        if entry.is_async:
            from model_tools import _run_async
            return _run_async(entry.handler(args, **kwargs))
        return entry.handler(args, **kwargs)
    except Exception as e:
        return json.dumps({"error": f"Tool execution failed: {type(e).__name__}: {e}"})
```



Note

设计洞察：

- **异常封装**：所有异常被捕获并格式化为 {"error": "..."} JSON 字符串返回。这一设计将异常处理边界固定在 registry 层，确保 Agent 不会因工具异常而崩溃。
- **统一字符串返回值**：无论工具返回什么类型，最终都序列化为字符串，简化 Agent Loop 的消息构造。但这里有一个潜在问题——如果 handler 返回的是字典，被 json.dumps() 后再传给 LLM，LLM 会看到双重序列化的字符串，可能影响解析。
- **异步透明桥接**：_run_async 从 registry 层调用而非 handler 层，保证了 handler 作者不需要关心同步/异步问题。

deregister() — 动态工具卸载

```
def deregister(self, name: str) -> None:
    entry = self._tools.pop(name, None)
    if entry.toolset in self._toolset_checks and not any(
        e.toolset == entry.toolset for e in self._tools.values()
    ):
        self._toolset_checks.pop(entry.toolset, None)
```



MCP 动态工具发现的核心机制。当 MCP 服务器工具列表变化时，整个工具集被清除后重新注册 ("nuke-and-repave"策略)。这是一种简单粗暴但有效的方案，避免了增量更新的复杂性。

3.3 查询接口

方法	用途
get_all_tool_names()	列举所有工具
get_schema(name)	获取原始 Schema (绕过 check_fn)
get_toolset_for_tool(name)	查询工具所属 toolset
get_emoji(name)	获取 UI emoji
is_toolset_available(toolset)	检查工具集可用性
check_tool_availability()	批量检查, 返回 (可用列表, 不可用详情)

4. 工具集系统 (Toolsets)

4.1 分层结构设计

Toolsets 通过 tools + includes 的二层结构，实现了一种轻量级的**工具组合代数**：

```
TOOLSETS = {
    "web": {
        "description": "Web research...",
        "tools": ["web_search", "web_extract"],
        "includes": []
    },
    "safe": {
        "description": "Safe toolkit...",
        "tools": ["mixture_of_agents"],
        "includes": ["web", "vision", "image_gen"] # 组合其他 toolsets
    },
}
```



直接工具 (tools) 是叶子节点，**包含关系 (includes)** 允许工具集互相引用，形成**钻石依赖**：

```
safe toolset
├─ tools: [mixture_of_agents]
└─ includes: [web, vision, image_gen]
    │
    ├─ web: [web_search, web_extract]
    └─ vision: [vision_analyze]
```



Note

乐观解析优于严格验证：resolve_toolset() 对循环依赖采取静默跳过策略，返回空列表而非抛出异常。代码注释明确区分了两者：钻石依赖 (safe → web → file 和 debugging → file 殊途同归) 是合法且正常的，静默跳过避免误报；真正的循环引用则被视为“小概率事件，安全跳过”。这种设计选择了可用性 (不因配置错误而崩溃) 而非正确性 (不警告循环)。与之形成对比的是一些语言的模块系统——它们在检测到循环时直接拒绝加载，这更安全但也更脆弱。在工具集配置场景中，Hermes 的选择是合理的，因为 toolset 定义错误应该由测试和 CI 捕获，而非运行时崩溃。

4.2 resolve_toolset() 的递归解析

```
def resolve_toolset(name: str, visited: Set[str] = None) -> List[str]:
    if name in {"all", "*"}: # 全局别名
        all_tools: Set[str] = set()
        for toolset_name in get_toolset_names():
            resolved = resolve_toolset(toolset_name, visited.copy())
            all_tools.update(resolved)
        return list(all_tools)

    if name in visited: # 循环检测
        return [] # 静默跳过

    visited.add(name)
    toolset = TOOLSETS.get(name)
    tools = set(toolset.get("tools", []))
    for included_name in toolset.get("includes", []):
```

```
tools.update(resolve_toolset(included_name, visited))

return list(tools)
```

设计洞察：

- **"all"/"" 别名**：使用特殊字符串作为“全部工具”的别名，并采用 `visited.copy()` 为每个分支创建独立的访问历史。这一细节至关重要——如果不 `copy`，同一工具集在不同路径中被第二次遇到时会触发误报循环。
- **静默循环跳过**：遇到循环时返回空列表而非抛出异常。注释解释道：钻石依赖是合法的（工具已在另一路径收集），真正的小概率循环也可以安全跳过。这是一种**乐观解析**策略，避免了循环检测破坏正常功能。
- **`visited.copy()` 在 "all" 别名中的特殊处理**：确保全局解析时，`visited` 集合在每条分支上独立，防止跨分支污染。

4.3 平台特定工具集

代码中定义了 13 个 messaging platform 工具集（telegram、discord、whatsapp、slack 等），它们**共享同一个 `_HERMES_CORE_TOOLS` 列表**：

```
_HERMES_CORE_TOOLS = [
    "web_search", "web_extract",
    "terminal", "process",
    "read_file", "write_file", "patch", "search_files",
    "vision_analyze", "image_generate",
    "mixture_of_agents",
    "skills_list", "skill_view", "skill_manage",
    "browser_navigate", "browser_snapshot", ...,
    "todo", "memory",
    "session_search", "clarify",
    "execute_code", "delegate_task",
    "cronjob",
    "send_message",
    "honcho_context", "honcho_profile", ...,
    "ha_list_entities", "ha_get_state", ...,
]
```

messaging 平台之间的安全策略差异（如 `terminal` 工具的危险命令审批机制）通过 `check_fn` 和 `dangerous command approval` 在 handler 层实现，而非 `toolset` 层。这保持了 `toolset` 的纯工具分组语义。

4.4 插件工具集的动态注册

```
def _get_plugin_toolset_names() -> Set[str]:
    return {
        entry.toolset
        for entry in registry._tools.values()
        if entry.toolset not in TOOLSETS
    }
```

这一机制允许插件在**运行时**向 `TOOLSETS` 字典中注入新的工具集名称。`get_all_toolsets()` 和 `validate_toolset()` 都能感知这些动态添加的条目，使插件提供的工具集与内置工具集在接口层面完全平等。

Note

"运行时注册"的实际含义：这里的“注册”是 Python 进程层面的能力，而非 **LLM 可感知的动态发现机制**。真正能在运行中通知 LLM 工具变更的是 **MCP 的 `notifications/tools/list_changed` 事件**：Hermes 的 MCP 客户端订阅了此事件，会触发 `deregister + 重新注册`，从而在下次 LLM API 调用时重新下发 Schema。Hermes 本地插件的注册则没有这个机制，工具集变更需要重启进程才能生效。

这种设计可能是考虑到 KV 缓存以及策略一致性的问题：突然告诉它“你多了 X 工具”，它会面临一个认知冲突：是继续当前计划（不用 X），还是重新规划（用 X）？这可能导致它做出与已执行步骤矛盾的新决策，特别是在 agent loop 正在进行多步操作时

5. 工具编排层（`model_tools.py`）

5.1 工具发现机制

```
def _discover_tools():
    _modules = [
        "tools.web_tools",
        "tools.terminal_tool",
        "tools.file_tools",
        ...
    ]
    for mod_name in _modules:
        try:
            importlib.import_module(mod_name)
        except Exception as e:
            logger.warning("Could not import tool module %s: %s", mod_name, e)
```

设计洞察：

- **静默失败策略**：可选工具（如 `fal_client` 未安装导致的 `image_generation_tool` 导入失败）不会使整个系统崩溃。这在开发实验性工具时非常有用，但也会掩盖真实的依赖问题。
- **硬编码模块列表**：工具模块列表维护在 `model_tools.py` 中，新增工具模块需要修改此列表。虽然 `toolsets.py` 能动态感知已注册的工具，但发现阶段仍需要显式声明。这一限制在实践中是可以接受的。

5.2 异步桥接的三层设计

`_run_async()` 函数处理同步调用异步工具的复杂场景：

```
def _run_async(coroutine):
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        loop = None

    if loop and loop.is_running():
        # 场景1: 在已有 async 上下文中 → 启动新线程
        import concurrent.futures
        with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
            future = pool.submit(asyncio.run, coroutine)
            return future.result(timeout=300)

    if threading.current_thread() is not threading.main_thread():
        # 场景2: 在 worker 线程中 → 使用线程本地持久 loop
        worker_loop = _get_worker_loop()
        return worker_loop.run_until_complete(coroutine)

    # 场景3: 在主线程同步上下文中 → 使用全局持久 loop
```

```
tool_loop = _get_tool_loop()
return tool_loop.run_until_complete(coro)
```



三重场景的处理逻辑:

场景	触发条件	解决方案	为什么
async 上下文中	loop.is_running() 为 True	ThreadPoolExecutor 新线程	不阻塞当前 event loop
worker 线程中	非主线程	线程本地持久 loop	避免 asyncio.run() 的 create/destroy 生命周期导致的 GC 问题
主线程同步中	默认路径	全局持久 loop	复用 loop 避免 "Event loop is closed"

5.3 Schema 动态修补

```
# 针对 execute_code 的动态 schema 修补
if "execute_code" in available_tool_names:
    sandbox_enabled = SANDBOX_ALLOWED_TOOLS & available_tool_names
    dynamic_schema = build_execute_code_schema(sandbox_enabled)
    # 替换原 schema

# 针对 browser_navigate 的描述清理
if "web_search" not in available_tool_names:
    # 从 browser_navigate 描述中移除 "prefer web_search" 字样
```



Note

设计意图: 防止工具名称幻觉 (tool name hallucination) —— LLM 在看到 Schema 中提及某个工具名时, 会倾向于调用它, 即使该工具实际不可用。

6. 工具调用解析器 (Tool Call Parsers)

6.1 问题背景

不同模型家族对工具调用的输出格式差异巨大:

模型	格式
Hermes 定制微调	<tool_call>{"name": "...", "arguments": {...}}</tool_call>
Llama	多种 JSON 结构
Qwen	特殊 XML 标签
Mistral	专用格式
DeepSeek	V3 / V3.1 不同格式
GLM	GLM-4.5 / GLM-4.7 不同格式
Kimi	K2 格式

6.2 解析器注册机制

```
class ToolCallParser(ABC):
    @abstractmethod
    def parse(self, text: str) -> ParseResult:
        raise NotImplementedError

PARSER_REGISTRY: Dict[str, Type[ToolCallParser]] = {}

def register_parser(name: str):
    def decorator(cls):
        PARSER_REGISTRY[name] = cls
        return cls
    return decorator

@register_parser("hermes")
class HermesToolCallParser(ToolCallParser):
    ...
```



6.3 Hermes Parser 示例

```
PATTERN = re.compile(
    r"<tool_call>\s*(.*?)\s*</tool_call>|<tool_call>\s*(.*)", re.DOTALL
)

def parse(self, text: str) -> ParseResult:
    if "<tool_call>" not in text:
        return text, None

    matches = self.PATTERN.findall(text)
    tool_calls = []
    for match in matches:
        raw_json = match[0] if match[0] else match[1]
        if not raw_json.strip():
            continue
        tc_data = json.loads(raw_json)
        tool_calls.append(ChatCompletionMessageToolCall(
            id=f"call_{uuid.uuid4().hex[:8]}",
            type="function",
            function=Function(
                name=tc_data["name"],
                arguments=json.dumps(tc_data.get("arguments", {})),
            ),
        ))

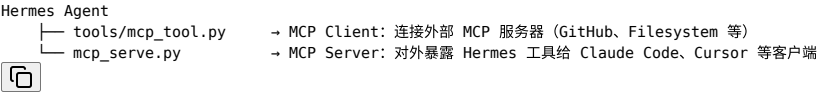
    content = text[:text.find("<tool_call>")].strip()
    return content if content else None, tool_calls
```



7. MCP（模型上下文协议）支持

7.1 架构定位

MCP 支持采用双向镜像架构：



这使得 Hermes 既可以是 MCP 工具的消费者，也可以是 MCP 工具的提供者。

7.2 MCP 客户端核心设计

传输层支持

```
# Stdio 传输: subprocess 方式
from mcp.client.stdio import stdio_client
async with stdio_client(config) as (read, write):
    async with ClientSession(stdio_interface) as session:
        ...

# HTTP 传输: 远程服务器
from mcp.client.streamable_http import streamablehttp_client
async with streamablehttp_client(url, headers=...) as (read, write, get_error):
    ...
```

命令解析增强

```
def _resolve_stdio_command(command: str, env: dict) -> tuple[str, dict]:
    """将 bare 命令 (npx/npm/node) 解析为绝对路径"""
    if os.sep not in resolved_command:
        which_hit = shutil.which(resolved_command, path=path_arg)
        if which_hit:
            resolved_command = which_hit
```

7.3 安全设计

环境变量过滤

```
_SAFE_ENV_KEYS = frozenset({
    "PATH", "HOME", "USER", "LANG", "LC_ALL", "TERM", "SHELL", "TMPDIR",
})

def _build_safe_env(user_env: Optional[dict]) -> dict:
    env = {}
    for key, value in os.environ.items():
        if key in _SAFE_ENV_KEYS or key.startswith("XDG_"):
            env[key] = value
    if user_env:
        env.update(user_env)
    return env
```

仅传递白名单中的环境变量 + 用户显式配置的变量，防止 API keys/tokens 泄露到 MCP subprocess 中。

凭证清理

```
_CREDENTIAL_PATTERN = re.compile(
    r"ghp_[A-Za-z0-9_]{1,255}"
    r"|sk-[A-Za-z0-9_]{1,255}"
    r"|Bearer\s+\S+"
    r"|token=[^\s&,;\"']{1,255}"
    ...
)
```

错误消息中的凭证模式被替换为 [REDACTED]，防止 MCP 服务器的错误响应暴露敏感信息。

7.4 Sampling 机制

MCP 服务器可以主动请求 LLM 采样（sampling/createMessage），SamplingHandler 实现：

```
class SamplingHandler:
    def __init__(self, server_name: str, config: dict):
        self.max_rpm = config.get("max_rpm", 10) # 每分钟请求数
        self.max_tool_rounds = config.get("max_tool_rounds", 5) # 工具循环轮次上限
        self.model_override = config.get("model") # 模型覆盖
        self.allowed_models = config.get("allowed_models", []) # 白名单

    def _check_rate_limit(self) -> bool:
        # 滑动窗口限速器
        now = time.time()
        window = now - 60
        self._rate_timestamps[:] = [t for t in self._rate_timestamps if t > window]
        ...
```

多层次防护：

- 速率限制：防止 MCP 服务器耗尽 LLM 配额
- 模型白名单：仅允许预批准的模型处理 Sampling 请求
- 工具循环上限：防止 Sampling 请求触发无限工具调用循环
- 每实例状态：SamplingHandler 的所有状态都是实例级别的（_rate_timestamps、metrics），没有模块级全局状态，支持多 MCP 服务器独立运行

7.5 重连与容错

```
_MAX_RECONNECT_RETRIES = 5
_MAX_BACKOFF_SECONDS = 60
```



采用**指数退避重连**，最长等待 60 秒。错误消息解析函数 `_format_connect_error()` 递归遍历嵌套异常链，提取最内层的 `FileNotFoundError`，给出可操作的错误提示（如 "missing executable 'npx'"）。

Note

MCP 异步守护线程的线程安全性：MCP client (`mcp_tool.py`) 在一个守护线程中运行独立的 `event loop`，所有工具调用通过 `run_coroutine_threadsafe()` 从外部线程安全地调度到这个 `loop` 上执行。这解决了工具系统（同步入口）与 MCP session（异步 API）的运行模型冲突——不需要将整个工具层异步化，只需隔离 MCP 这一个异步子系统。这是一种经典的**线程隔离解耦模式**：将最复杂的异步部分封装在一个线程中，对外呈现同步接口。缺点是增加了一个常驻线程的开销，以及守护线程崩溃时的恢复复杂性。

8. 插件系统

8.1 插件发现三源

```
# 1. 用户插件: ~/.hermes/plugins/<name>/
# 2. 项目插件: ./hermes/plugins/<name>/
# 3. Pip 插件: packages with hermes_agent.plugins entry-point
```



8.2 PluginContext 门面

```
class PluginContext:
    def register_tool(self, name, toolset, schema, handler, ...):
        from tools.registry import registry
        registry.register(...)

    def register_hook(self, hook_name: str, callback):
        self._manager._hooks.setdefault(hook_name, []).append(callback)

    def inject_message(self, content, role):
        """将消息注入当前对话上下文"""
        self._manager._injected_messages.append({"role": role, "content": content})
```



Note

插件不应该知道底层注册中心的具体实现。PluginContext 提供了**门面模式 (Facade Pattern)**，将 `registry`、`hooks`、`message injection` 等多个子系统封装在单一接口后。

8.3 生命周期钩子

```
VALID_HOOKS = {
    "pre_tool_call",    # 工具执行前
    "post_tool_call",   # 工具执行后
    "pre_llm_call",     # LLM 调用前
    "post_llm_call",    # LLM 调用后
    "on_session_start", # 会话开始时
    "on_session_end",   # 会话结束时
}
```



钩子机制提供了**中间件层**的核心能力，使得插件可以在不修改核心代码的情况下：

- 在工具执行前验证参数（如安全扫描）
- 在工具执行后记录审计日志
- 在 LLM 调用前注入系统提示词
- 在 LLM 调用后修改响应

8.4 禁用机制

```
def _get_disabled_plugins() -> set:
    config = load_config()
    disabled = config.get("plugins", {}).get("disabled", [])
    return set(disabled)
```



通过 `config.yaml` 文件中的 `plugins.disabled` 字段，用户可以在不停机的情况下禁用问题插件。

Note

渐进式技能加载的工程取舍：Skills 系统采用渐进式披露（Progressive Disclosure）策略：`skills_list` 仅返回元数据（名称 ≤64 字符，描述 ≤1024 字符），`skill_view` 才加载完整指令内容。这与 MCP 将所有工具 schema 一次性暴露的设计形成对比——MCP 的方式对 LLM 更友好（无需额外工具调用），但 token 成本高；Hermes 的方式 token 高效但增加交互轮次。YAML frontmatter 同时充当 manifest 和安全策略载体（platforms 限制加载平台，prerequisites 声明运行时依赖），无需额外配置层。这种将元数据与内容共置的设计，减少了技能描述与实际代码之间的漂移风险。

9. 代码执行沙箱

9.1 沙箱工具集注入

```
if "execute_code" in available_tool_names:
    sandbox_enabled = SANDBOX_ALLOWED_TOOLS & available_tool_names
    dynamic_schema = build_execute_code_schema(sandbox_enabled)
```



`execute_code` 工具允许 LLM 生成 Python 代码并执行。这些生成的代码可以调用其他 Hermes 工具，从而**大幅减少 LLM 交互轮次**（一个代码块中可以依次调用多个工具）。

关键设计：代码执行环境中的可用工具由**外部传入的工具集**决定，而非全局注册表。这使得 Agent Loop 可以精确控制子代理的工具权限。

9.2 多种执行后端

代码执行支持多种隔离环境：

```
tools/environments/
├── base.py          # 抽象基类
├── docker.py        # Docker 容器隔离
├── local.py         # 直接执行（仅 dev 模式）
├── daytona.py       # Daytona 商业沙箱
├── modal.py         # Modal 云函数
├── ssh.py           # 远程 SSH 执行
├── singularity.py   # Singularity 容器
└── persistent_shell.py # 持久化 shell 会话
```



这一多后端设计反映了 Hermes Agent 的**生产级野心**：不同的部署场景需要不同的隔离级别，从开发阶段的本地直接执行，到生产环境中的 Docker/Modal 隔离。

Note

沙箱工具注入：显式参数化而非全局查找：execute_code 工具的可用工具集由 enabled_tools 参数在调用点显式注入，而不是在 handler 内部查询全局注册表。这一设计选择至关重要：它防止了子代理通过执行任意代码间接调用父会话中未被授权的工具。如果使用全局查询，delegate_task 启动的受限子代理可能会继承父会话的完整工具集，造成权限逃逸。将权限边界固定在调用栈上，是一种轻量级的**能力空间隔离**——无需进程级隔离，只需参数传递正确。

10. 生命周期钩子与拦截器

10.1 Agent Loop 工具拦截

```
_AGENT_LOOP_TOOLS = {"todo", "memory", "session_search", "delegate_task"}

def handle_function_call(...):
    if function_name in _AGENT_LOOP_TOOLS:
        return json.dumps({"error": f"{function_name} must be handled by the agent loop"})
```



这些工具被 registry 注册，但分发时返回错误，强制由 Agent Loop 亲自处理。原因：**这些工具需要访问 Agent Loop 层的状态**（TodoStore、MemoryStore 等），通过 registry 分发会丢失上下文。

这是一个**框架强制而非惯例约束**的设计：用错误消息而非代码路径来防止直接调用。理论上直接调用 registry 仍然可行，但 LLM 会收到明确的错误消息。

10.2 读写工具连续性检测

```
_READ_SEARCH_TOOLS = {"read_file", "search_files"}

def handle_function_call(...):
    if function_name not in _READ_SEARCH_TOOLS:
        notify_other_tool_call(task_id)
```



连续的文件读取操作（超过阈值）会触发警告，防止 Agent 在循环中不断读取文件而不进行修改。这是一种**隐式的循环检测**——不是直接检测循环，而是检测“有意行为”（修改文件）与“惰性行为”（连续读取）的比例失衡。

11. 工具集分发系统

11.1 RL 训练中的工具概率分布

```
DISTRIBUTIONS = {
    "default": {
        "description": "All available tools, all the time",
        "toolsets": {
            "web": 100, "vision": 100, "image_gen": 100,
            "terminal": 100, "file": 100, "moa": 100, "browser": 100
        }
    },
    "image_gen": {
        "description": "Heavy focus on image generation...",
        "toolsets": {
            "image_gen": 90, "vision": 90, "web": 55,
            "terminal": 45, "moa": 10
        }
    },
}
```



在强化学习训练阶段，不同的工具集分布使 Agent 学习到不同场景下的工具使用策略。“image_gen”分布降低“moa”（推理工具）的概率，将更多训练预算分配给图像生成相关的场景。

Note

RL 训练分布与生产环境的隐性不匹配：训练时使用的 toolset 概率分布直接塑造了 Agent 的行为策略——它学会在特定工具可用时优先选择它们。但在生产部署时，如果实际启用的工具集与训练分布不同（例如 default 分布中 terminal 权重为 100，但部署时禁用了 terminal），Agent 可能表现与训练时不一致。这是一种**部署环境偏移（distribution shift）**问题，在 RL-based Agent 中比监督学习更隐蔽：模型不只是生成不同的输出，而是在不同工具可用性条件下调用了不同的工具组合。toolset_distributions 的引入说明设计者意识到了这一点——但如何保证训练分布与生产配置持续同步，仍是一个需要手动维护的工程约束。

12. 架构决策中的权衡

12.1 单例 Registry vs 依赖注入

选择：模块级单例 registry = ToolRegistry()

权衡：

优点	缺点
无需 DI 容器，简单直接	测试时需要 mock 或 patch 全局状态
全局访问无性能开销	MCP 等多进程场景中 registry 状态不共享
导入即用的开发者体验好	

在 CLI 工具（单进程、同步）的场景下，单例是完全合理的选择。但对于 MCP 多服务器架构，每个 MCPServerTask 运行在独立的 event loop 上，共享同一个 registry 实例——虽然目前工作正常，但未来多进程扩展时需要重新设计。

12.2 静默失败 vs 严格检查

工具模块导入失败时只发出 WARNING 而非 ERROR：

```
try:
    importlib.import_module(mod_name)
except Exception as e:
    logger.warning("Could not import tool module %s: %s", mod_name, e)
```



权衡：

- **优点：**可选依赖（如 fal_client 图像生成）缺失不会导致系统无法启动；实验性工具可以在不影响主系统的情况下引入。
- **缺点：**缺失工具的根因（未安装、导入错误、版本冲突）被隐藏；Warning 级别日志在默认配置下可能被完全忽略，导致用户不知道某工具不可用。

12.3 check_fn 粒度

当前 check_fn 是 **toolset 级别**的可用性检查，不是工具级别的：

```
if check_fn and toolset not in self._toolset_checks:
    self._toolset_checks[toolset] = check_fn
```



这意味着同一 toolset 的所有工具共享同一个 check_fn。如果一个 toolset 包含 5 个工具，但只有 2 个需要某个 API key，当前设计会让这 2 个工具的 check_fn 同时控制全部 5 个工具的可用性。实践中 Hermes 的 toolset 划分足够细（每个 toolset 通常 1-3 个工具），这一限制影响不大，但限制了工具集的进一步组合。

12.4 工具 Schema 的版本管理

当前没有 Schema 版本管理机制——当工具参数变更时，Schema 随代码更新而更新，LLM 通过 API 调用获得最新 Schema。这在持续部署环境中可行，但在模型微调场景下可能有问题：旧版本微调模型可能生成了与新版 Schema 不兼容的参数。

13. 安全设计分析

13.1 纵深防御

Hermes Agent 的工具系统实现了多层次安全控制：

层级	机制	防护目标
传输安全	MCP 环境变量过滤	凭证泄露到 subprocess
内容安全	凭证清理正则	错误消息中的凭证暴露
平台安全	check_fn	平台特定的工具可用性
交互安全	dangerous command approval messaging	平台的危险命令
沙箱安全	多后端隔离	代码执行隔离
插件安全	disabled list	有问题的插件禁用
Schema 安全	动态交叉引用清理	工具名称幻觉

13.2 凭证清理的局限性

正则清理虽然覆盖了常见凭证格式，但仍存在盲点：

- 环境变量中的凭证（如 \$GITHUB_TOKEN）不会被清理
- 非标准格式的自定义 API keys
- MCP 服务器响应中的结构化凭证（JSON body 中的 token）

这是一个**已知的不完美方案**：完美的凭证清理需要语义理解，而正则只能覆盖模式匹配。设计者选择了“尽可能多地覆盖常见模式”而非“尝试完美但可能过度清理合法内容”。

Note

Tirith：二进制扫描与正则扫描的双层架构：Hermes 有两套互补的安全扫描机制：skills_guard.py 使用纯 Python 正则静态分析技能文件，针对已知威胁模式（数据外泄、提示注入等）；tirith_security.py 调用外部 Rust 二进制工具进行更深入的语义扫描。两者的策略一致：**fail-open（失败后放行）**——tirith 超时或不可用时只记录 WARNING，不阻止操作。这在安全性和可用性之间选择了实用路线：宁可放过可疑内容，也不让安全工具本身成为单点故障。auto-install 机制（通过 SHA-256 校验和下载，配合 cosign 供应链验证）确保 tirith 始终可用，但也意味着安全边界依赖于 Hermes 发布的 tirith 版本未被篡改——这是一个外部二进制供应链风险。

参考文件索引

文件	行数	描述
tools/registry.py	276	核心注册中心
toolsets.py	642	工具集定义与解析
model_tools.py	473	工具编排层
tools/mcp_tool.py	~600+	MCP 客户端
mcp_serve.py	—	MCP 服务器
hermes_cli/plugins.py	~200+	插件系统
environments/tool_call_parsers/	—	11 个 Parser 实现
tools/code_execution_tool.py	—	代码执行沙箱
toolset_distributions.py	—	RL 工具分发
environments/agent_loop.py	—	Agent 主循环